



Proyecto Final: Soft Kitty Warm Kitty Pets Market

Proyecto Final

Estudiantes:

Ricardo Andrei Borge Carvajal

Jose Andres Fernandez Gutierrez

Ruben Antonio Cruz Gutierrez

Daniel Mendez Zeledon

Universidad CENFOTEC

Principios de Programacion 2

Profesor: Robert Alexis Castro Mendez

Abril 2025

# Tabla de Contenido

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>Tabla de Contenido.....</b>                        | <b>2</b>  |
| <b>Introducción.....</b>                              | <b>3</b>  |
| <b>Diagrama de Clases.....</b>                        | <b>4</b>  |
| <b>Clases definidas.....</b>                          | <b>5</b>  |
| Clase Productos e Inventario.....                     | 6         |
| Métodos definidos para Inventario.....                | 7         |
| inicializarInventario().....                          | 7         |
| agregarProducto().....                                | 7         |
| buscarProducto().....                                 | 8         |
| actualizarStock().....                                | 8         |
| modificarProducto().....                              | 8         |
| Ejemplos de la Gestión de Inventario.....             | 9         |
| Ejemplo del menú de Agregar Producto.....             | 9         |
| Ejemplo del menú de Modificar Producto.....           | 10        |
| Clase Ventas.....                                     | 13        |
| Métodos definidos para Ventas.....                    | 14        |
| Agregar Producto a la Factura.....                    | 14        |
| Generar la Factura.....                               | 14        |
| Guardar el Inventario después de una Venta.....       | 15        |
| Ejemplos de la gestión de ventas.....                 | 15        |
| Gestión de la venta.....                              | 16        |
| Clase CargaInformacion.....                           | 18        |
| Métodos definidos para la Clase CargaInformacion..... | 19        |
| Clase Cliente y ClienteGestiones.....                 | 20        |
| Metodos definidos en ClienteGestiones.....            | 22        |
| Clase Usuario y UsuarioGestiones.....                 | 23        |
| Métodos definidos en UsuarioGestiones.....            | 24        |
| Clase Sistema.....                                    | 25        |
| Metodos definidos en Clase Sistema.....               | 26        |
| Ejemplos del uso del sistema.....                     | 26        |
| Ejemplo Usuario tipo GERENTE.....                     | 26        |
| Ejemplo Usuario tipo VENDEDOR.....                    | 28        |
| Clase FastOptionPane.....                             | 29        |
| <b>Link al video de presentación.....</b>             | <b>30</b> |

# Introducción

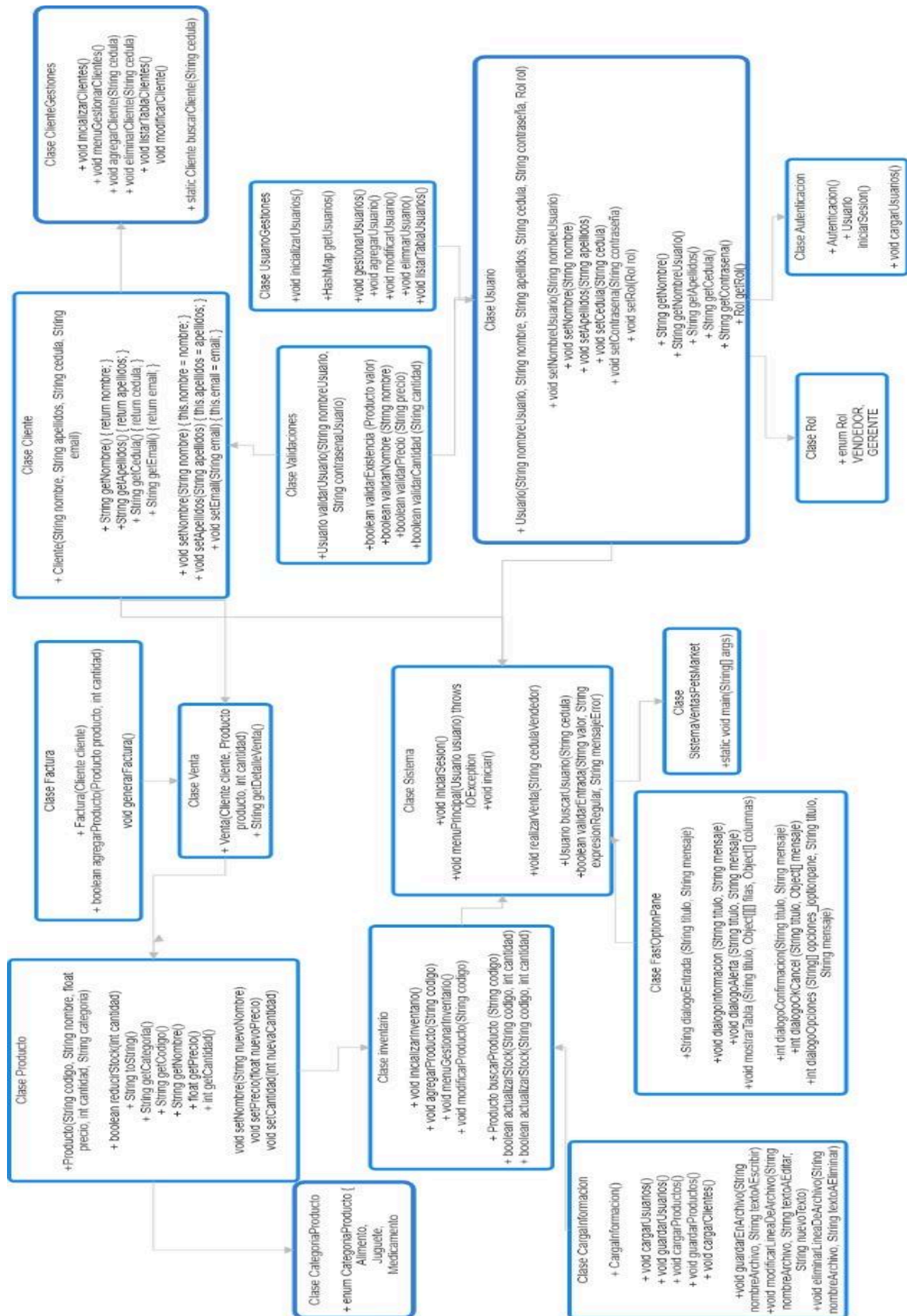
La industria de los Pet Shops ha crecido a pasos agigantados en Costa Rica, debido a que mas y mas personas deciden incluir en su familia a alguna mascota, cambiando a su vez el paradigma de que a los animales se les dan las sobras por una visión clara, donde los animales que tienen, merecen los mejores cuidados, la mejor alimentación y el mejor entretenimiento para el disfrute total de ellos y sus familias.

Con esto, las empresas dedicadas a la venta de artículos para mascotas se ven en la necesidad de mejorar sus sistemas de inventarios, usuarios, clientes y ventas, con el fin de dar un mejor servicio a sus clientes, así como facilitar el control de los recursos y flujo de dinero que tengan, como en cualquier otro lugar comercial.

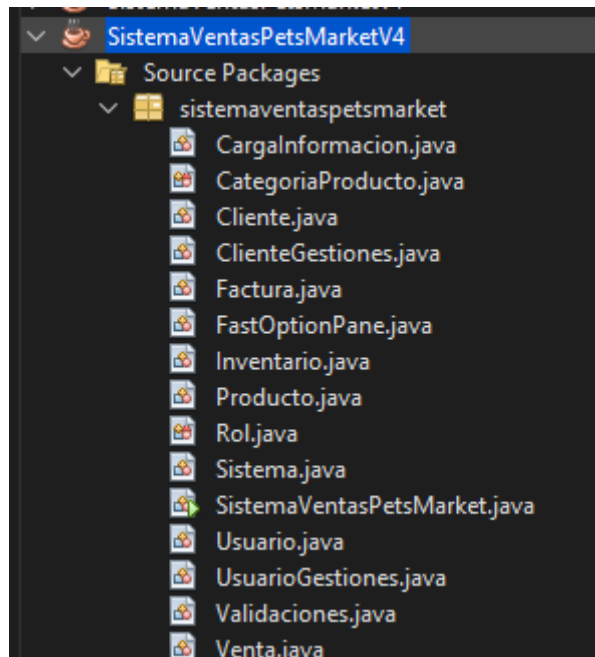
Nuestra empresa, Bow&Arrow Technologies, se encarga de dar servicios de software, basados en las necesidades específicas de cada cliente. En el presente caso, hemos sido contratados para elaborar un pequeño sistema, donde la tienda Soft Kitty Warm Kitty Pets Shop pueda tener gestion y control sobre las ventas e inventarios de sus productos, asi como tener la opción de tener diferentes usuarios para el uso del sistema y una lista de clientes.

Dado que la empresa cuenta con poco presupuesto, se le ofrece la opción de elaborar el sistema en un formato simple, con ventanas emergentes, sin quitar la seguridad, facilidad de uso y confiabilidad del mismo. El mismo será implementado usando el lenguaje Java, debido a su robustez y estabilidad, haciendo uso de la Programación Orientada a Objetos, con el fin de distribuir cada parte del sistema en pequeños bloques que interactúan para dar sentido al sistema.

# Diagrama de Clases



## Clases definidas



Con el proyecto ya avanzado, hemos modificado varias de las clases con el fin de encapsular más los diferentes procesos de cada una de las áreas. A lo largo del curso, se han aprendido nuevas formas y mejores maneras de organizar el código, por lo que se han implementado clases como “CargaInformacion”, la cual se utiliza para cargar en el sistema, los diferentes archivos que se comportan como bases de datos. A su vez, los Usuarios, Productos y Clientes fueron encapsulados en su clase, donde a través de los gets y sets se obtienen su información, mientras que las clases llamadas “Gestiones” son las que se encargan de ejecutar las funciones para cada uno de los agentes que se interconectan. Al finalizar el proyecto, es posible que algunas clases deban ser eliminadas ya que se dejaron de usar.

## Clase Productos e Inventario

```
public class Producto {  
    private String codigo;  
    private String nombre;  
    private float precio;  
    private int cantidad;  
    private String categoria;  
  
    public Producto(String codigo, String nombre, float precio, int cantidad, String categoria) {  
        this.codigo = codigo;  
        this.nombre = nombre;  
        this.precio = precio;  
        this.cantidad = cantidad;  
        this.categoria = categoria;  
    }  
  
    public String toString() {  
        return codigo + "," + nombre + "," + precio + "," + cantidad + "," + categoria;  
    }  
  
    public String getCategoria() {  
        return categoria;  
    }  
  
    public String getCodigo() {  
        return codigo;  
    }  
  
    public float getPrecio() {  
        return precio;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public int getCantidad() {  
        return cantidad;  
    }  
  
    void setNombre(String nuevoNombre) {  
        this.nombre = nuevoNombre;  
    }  
  
    void setPrecio(float nuevoPrecio) {  
        this.precio = nuevoPrecio;  
    }  
}
```

La clase Producto define el objeto tipo Producto, con los argumentos necesarios para su creación. A su vez tiene el constructor necesario para ser utilizado y los gets y sets para el encapsulamiento de la información.

```

    */
    public class Inventario {
        //Inicializacion de Clases
        private FastOptionPane fop = new FastOptionPane();
        private Validaciones val = new Validaciones();
        private CargaInformacion carga = new CargaInformacion();
        //Inicializacion de Variables
        private HashMap<String, Producto> productosPorCodigo = new HashMap<>();
        private String nuevoNombre, precioString, cantidadString;
        private float nuevoPrecio;
        private int nuevaCantidad;

        private JTextField campoNombreF = new JTextField();
        private JTextField campoPrecioF = new JTextField();
        private JTextField campoCantidadF = new JTextField();
        private JTextField campoCategoriaF = new JTextField();

        public void inicializarInventario(){
            carga.cargarProductos();
            productosPorCodigo = carga.getInventarioCargado();
        }

        public void agregarProducto(String codigo) {
            Object[] opcionesProducto = new Object[]{"Por favor, ingrese los datos del producto", " ", "Nombre del Producto", campoNombreF, "Precio del Producto", campoPrecioF,
            "Cantidad del Producto", campoCantidadF, "Categoria de Producto", campoCategoriaF};
            int confirm = fop.dialogoOkCancel("Agregar Producto", opcionesProducto);
            if (confirm == 0){
                String nombre = campoNombreF.getText();
                float precio = Float.parseFloat(campoPrecioF.getText());
                int cantidad = Integer.parseInt(campoCantidadF.getText());
                String categoria = campoCategoriaF.getText();
                Producto producto = new Producto(codigo, nombre, precio, cantidad, categoria);
                productosPorCodigo.putIfAbsent(producto.getCodigo(), producto);
                fop.dialogoInformacion("Agregar Producto", "Producto agregado satisfactoriamente.");
                carga.guardarProductos();
            } else {
                fop.dialogoInformacion("Agregar Producto", "El producto no fue agregado.");
            }
        }
    }

```

A lo largo de todo el proyecto, se ha utilizado el Hashmap como método de memoria para la gestión de la información. La clase Inventario no es la excepción. Esta utiliza el hashmap para almacenar los productos. El HashMap está diseñado para organizar los productos por categorías, lo cual facilita la gestión y búsqueda de productos según su tipo. La clave del HashMap es el nombre de la categoría y el valor es una lista de productos dentro de esa categoría.

## Métodos definidos para Inventario

### inicializarInventario()

```

public void inicializarInventario(){
    carga.cargarProductos();
    productosPorCodigo = carga.getInventarioCargado()
}

```

Llama al método “cargarProductos” de la clase “CargaInformacion” para inicializar el hashmap con todos los productos y luego obtiene ese resultado y lo guarda en un nuevo hashmap llamado “productosPorCodigo”

### agregarProducto()

```

public void agregarProducto(String codigo) {
    Object[] opcionesProducto = new Object[]{"Por favor, ingrese los datos del producto", " ", "Nombre del Producto", campoNombreF, "Precio del Producto", campoPrecioF,
    "Cantidad del Producto", campoCantidadF, "Categoria de Producto", campoCategoriaF};
    int confirm = fop.dialogoOkCancel("Agregar Producto", opcionesProducto);
    if (confirm == 0){
        String nombre = campoNombreF.getText();
        float precio = Float.parseFloat(campoPrecioF.getText());
        int cantidad = Integer.parseInt(campoCantidadF.getText());
        String categoria = campoCategoriaF.getText();
        Producto producto = new Producto(codigo, nombre, precio, cantidad, categoria);
        productosPorCodigo.putIfAbsent(producto.getCodigo(), producto);
        fop.dialogoInformacion("Agregar Producto", "Producto agregado satisfactoriamente.");
        carga.guardarProductos();
    } else {
        fop.dialogoInformacion("Agregar Producto", "El producto no fue agregado.");
    }
}

```

Este método agrega un producto a una categoría específica dentro del HashMap. Si no existe una lista para la categoría del producto, se crea una nueva.

Aquí se usa `putIfAbsent` para asegurarse de que, si la categoría no existe, se crea automáticamente un Objeto para esa categoría, y luego se agrega el producto.

### **buscarProducto()**

```
public Producto buscarProducto(String codigo) {  
    Producto producto = productosPorCodigo.get(codigo);  
    if (producto != null && producto.getCodigo().equals(codigo)) {  
        return producto;  
    } else {  
        return null;  
    }  
}
```

Este método recorre todas las categorías en el HashMap y busca un producto por su código. Esto puede ser útil si no sabes en qué categoría se encuentra el producto, pero quieres buscarlo por su código único.

### **actualizarStock()**

Este método busca el producto por su código y reduce su cantidad en inventario, llamando al método `reducirStock()` del producto.

```
public boolean actualizarStock(String codigo, int cantidad) {  
    Producto producto = buscarProducto(codigo);  
    return producto != null && producto.reducirStock(cantidad);  
}
```

### **modificarProducto()**

Este método permite modificar los detalles de un producto (nombre, precio, cantidad). Busca el producto por su código y luego permite al usuario ingresar los nuevos datos.



```

public void modificarProducto(String codigo) {
    Producto producto = buscarProducto(codigo);
    String[] opcionesModificar = new String[]{"Nombre", "Precio", "Cantidad", "Categoria", "Salir"};
    int otroVal = 1;
    boolean activo = true;

    while (activo) {
        int seleccion = fop.dialogoOpciones(opcionesModificar, "Modificar Producto", "Seleccione la opcion que desea modificar");
        if (seleccion == JOptionPane.CLOSED_OPTION) break;

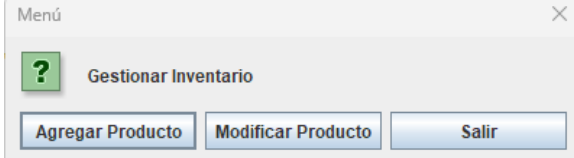
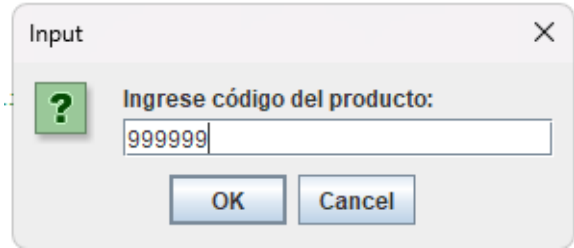
        switch (seleccion) {
            case 0:
                boolean valido = true;
                while (valido) {
                    nuevoNombre = fop.dialogoEntrada("Modificar Producto", "Ingrese nuevo nombre:");
                    valido = val.validarNombre(nuevoNombre);
                }
                int reintentar = fop.dialogoConfirmacion("Modificar Nombre", "Valor ingresado: " + nuevoNombre + "\nEl valor ingresado es correcto?");
                if (reintentar == 0) {
                    producto.setNombre(nuevoNombre);
                    fop.dialogoInformacion("Modificar Producto", "Nombre modificado satisfactoriamente");
                } else {
                    fop.dialogoInformacion("Modificar Producto", "Nombre no modificado");
                }
                otroVal = fop.dialogoConfirmacion("Modificar Producto", "Desea modificar otro valor?");
                if (otroVal == 1) {
                    activo = false;
                }
                break;
            case 1:
                valido = true;
                while (valido) {
                    precioString = fop.dialogoEntrada("Modificar Producto", "Ingrese nuevo precio:");
                    valido = val.validarPrecio(precioString);
                    nuevoPrecio = Float.parseFloat(precioString);
                    producto.setPrecio(nuevoPrecio);
                    fop.dialogoInformacion("Modificar Producto", "Precio modificado satisfactoriamente");
                }
                otroVal = fop.dialogoConfirmacion("Modificar Producto", "Desea modificar otro valor?");
                if (otroVal == 1) {
                    activo = false;
                }
                break;
        }
    }
}

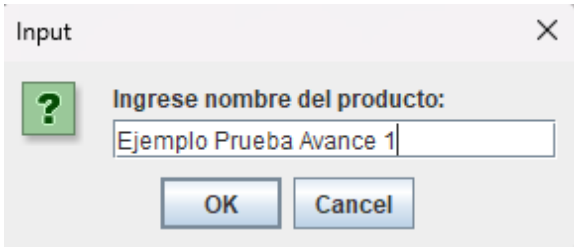
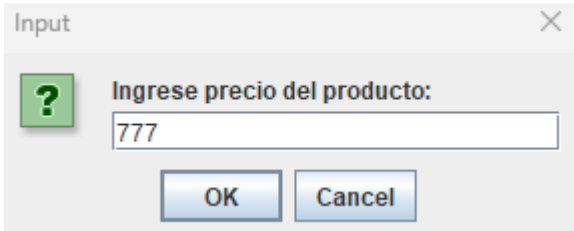
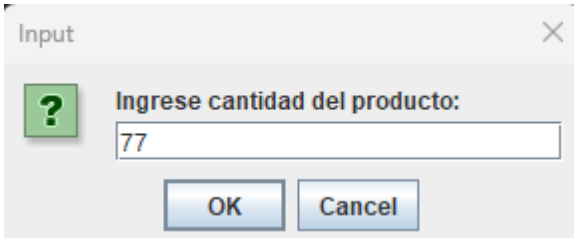
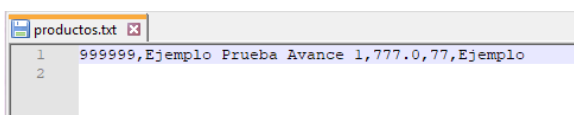
```

## Ejemplos de la Gestión de Inventario

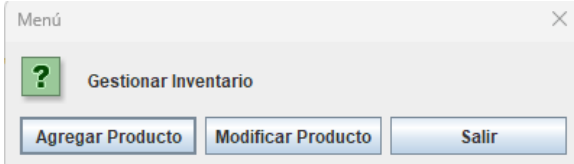
A continuación mostramos ejemplos de los botones de la gestión de inventario.

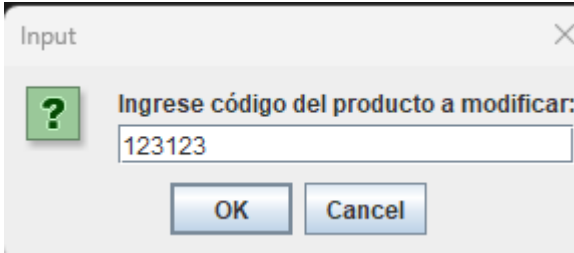
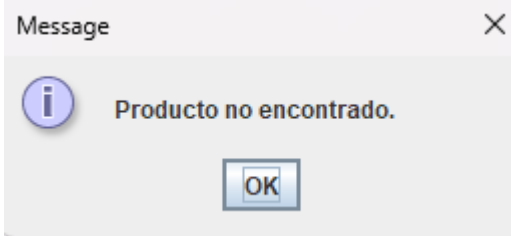
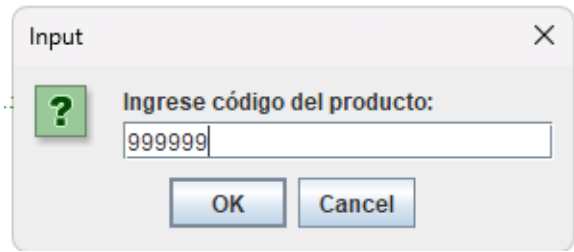
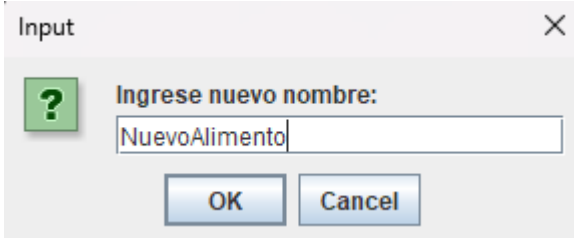
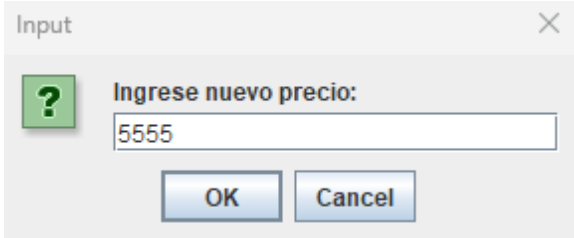
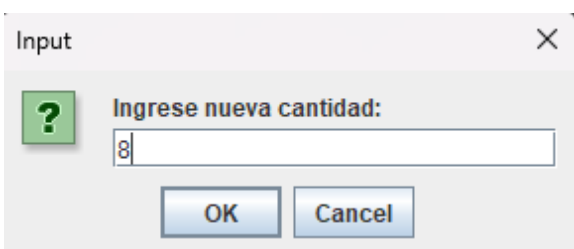
### Ejemplo del menú de Agregar Producto

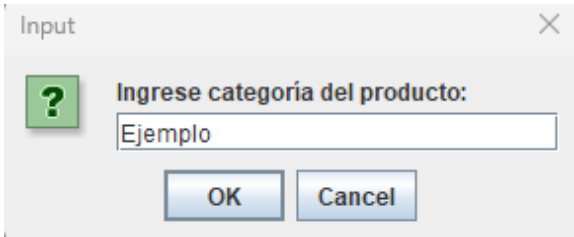
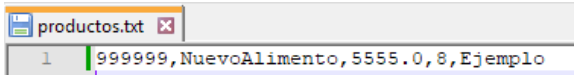
| Acción                                               | Imagen de Ejemplo                                                                    |
|------------------------------------------------------|--------------------------------------------------------------------------------------|
| Menú principal para GERENTE                          |  |
| Menú al ingresar en el botón de Gestionar Inventario |  |
| Agregar Producto                                     |  |

|                                                   |                                                                                      |
|---------------------------------------------------|--------------------------------------------------------------------------------------|
| Agregar Nombre del Producto                       |    |
| Agregar Precio del Producto                       |    |
| Agregar Cantidad del Producto                     |   |
| Agregar Categoría del Producto                    |  |
| Muestra del archivo donde se guarda el inventario |  |

### Ejemplo del menú de Modificar Producto

| Acción                                               | Imagen de Ejemplo                                                                    |
|------------------------------------------------------|--------------------------------------------------------------------------------------|
| Menú al ingresar en el botón de Gestionar Inventario |  |

|                                  |                                                                                                                                                                                                                                                                                                                        |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Modificar Producto               |  <p>Input dialog box titled "Input" with a close button (X). It contains a green question mark icon, the text "Ingrese código del producto a modificar:", a text input field containing "123123", and "OK" and "Cancel" buttons.</p> |
| Fallo por no existir el producto |  <p>Message dialog box titled "Message" with a close button (X). It contains a purple information icon, the text "Producto no encontrado.", and an "OK" button.</p>                                                                  |
| Modificar Producto               |  <p>Input dialog box titled "Input" with a close button (X). It contains a green question mark icon, the text "Ingrese código del producto:", a text input field containing "999999", and "OK" and "Cancel" buttons.</p>            |
| Modificación de Nombre           |  <p>Input dialog box titled "Input" with a close button (X). It contains a green question mark icon, the text "Ingrese nuevo nombre:", a text input field containing "NuevoAlimento", and "OK" and "Cancel" buttons.</p>           |
| Modificar Precio del Producto    |  <p>Input dialog box titled "Input" with a close button (X). It contains a green question mark icon, the text "Ingrese nuevo precio:", a text input field containing "5555", and "OK" and "Cancel" buttons.</p>                    |
| Modificar Cantidad del Producto  |  <p>Input dialog box titled "Input" with a close button (X). It contains a green question mark icon, the text "Ingrese nueva cantidad:", a text input field containing "8", and "OK" and "Cancel" buttons.</p>                     |

|                                                                             |                                                                                    |
|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <p>Agregar Categoría del Producto</p>                                       |  |
| <p>Muestra del archivo donde se guarda el inventario luego de modificar</p> |  |

## Clase Ventas

Para realizar la venta se utiliza el metodo realizarVenta() en el cual se verifica primero que el cliente esté registrado mediante la cédula, se le solicita el código del producto y luego la cantidad y después intenta agregar el producto a la factura, termina cuando el usuario ingresa “salir” y se genera la factura.

```
public void realizarVenta(String cedulaVendedor) {
    // Solicitar cédula del cliente
    String cedulaCliente = JOptionPane.showInputDialog("Ingrese cédula del cliente:");

    // Verificar si el usuario canceló la entrada
    if (cedulaCliente == null || cedulaCliente.isEmpty()) {
        JOptionPane.showMessageDialog(null, "Operación cancelada.");
        return; // Si el usuario cancela, salimos de la función
    }

    // Buscar cliente
    Cliente cliente = buscarCliente(cedulaCliente);
    if (cliente == null) {
        JOptionPane.showMessageDialog(null, "Cliente no encontrado.");
        return;
    }

    // Crear factura para el cliente
    Factura factura = new Factura(cliente);

    String opcion;
    do {
        // Solicitar código de producto o 'salir' para finalizar
        opcion = JOptionPane.showInputDialog("Ingrese el código del producto para vender (o 'salir' para finalizar la venta):");

        // Verificar si el usuario canceló o dejó el campo vacío
        if (opcion == null || opcion.isEmpty()) {
            JOptionPane.showMessageDialog(null, "Operación cancelada.");
            return; // Salir si el usuario cancela
        }

        // Si el código no es "salir", intentamos agregar el producto a la factura
        if (!opcion.equalsIgnoreCase("salir")) {
            Producto producto = inventario.buscarProducto(opcion);
            if (producto != null) {
                // Solicitar cantidad
                int cantidad = Integer.parseInt(JOptionPane.showInputDialog("Ingrese la cantidad:"));

                // Verificar si el usuario canceló la entrada de cantidad
                if (cantidad <= 0) {
                    JOptionPane.showMessageDialog(null, "Cantidad inválida.");
                    continue;
                }

                // Intentar agregar el producto a la factura
                if (factura.agregarProducto(producto, cantidad)) {
                    JOptionPane.showMessageDialog(null, "Producto agregado a la factura.");
                } else {

```

```
                } else {
                    JOptionPane.showMessageDialog(null, "No hay suficiente stock del producto: " + producto.getNombre());
                }
            } else {
                JOptionPane.showMessageDialog(null, "Producto no encontrado.");
            }
        }
    } while (!opcion.equalsIgnoreCase("salir")); // Terminar cuando el usuario ingrese "salir"

    // Después de salir del ciclo, generar la factura completa
    factura.generarFactura();
}
```

## Métodos definidos para Ventas

### Agregar Producto a la Factura

Cuando un cliente agrega productos a la factura, el sistema reduce la cantidad del producto en el inventario al confirmar la venta. Para hacer esto, se puede usar el método `actualizarStock()`.

```
public boolean actualizarStock(String codigo, int cantidad) {  
    Producto producto = buscarProducto(codigo);  
    return producto != null && producto.reducirStock(cantidad);  
}
```

```
public boolean reducirStock(int cantidad) {  
    if (this.cantidad >= cantidad) {  
        this.cantidad -= cantidad;  
        return true;  
    }  
    return false;  
}
```

### Generar la Factura

Una vez que se ha agregado los productos a la factura, se genera la factura al final de la venta.

```
public void generarFactura() {  
    StringBuilder detalles = new StringBuilder();  
    detalles.append("Factura para: ").append(cliente.getNombre()).append(" ").append(cliente.getApellidos())  
        .append("\nCédula: ").append(cliente.getCedula()).append("\n\n");  
  
    detalles.append("Productos comprados:\n");  
    for (Producto p : productos) {  
        detalles.append("Producto: ").append(p.getNombre())  
            .append(", Cantidad: ").append(p.getCantidad())  
            .append(", Precio unitario: ").append(p.getPrecio())  
            .append("\n");  
    }  
  
    detalles.append("\nTotal a pagar: ").append(this.total);  
    JOptionPane.showMessageDialog(null, detalles.toString());  
}
```

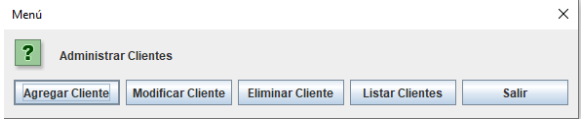
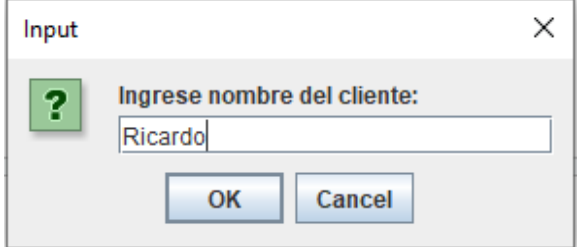
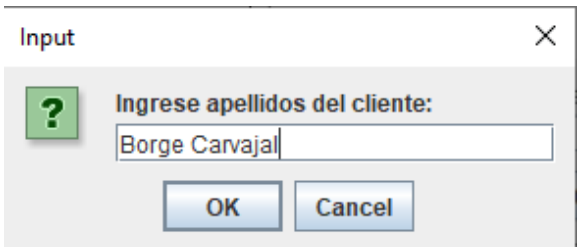
## Guardar el Inventario después de una Venta

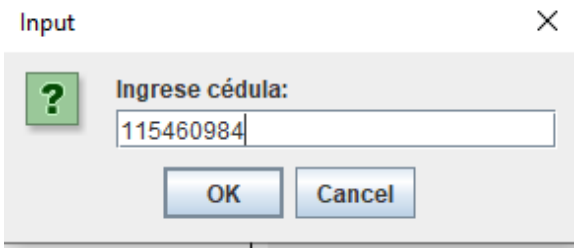
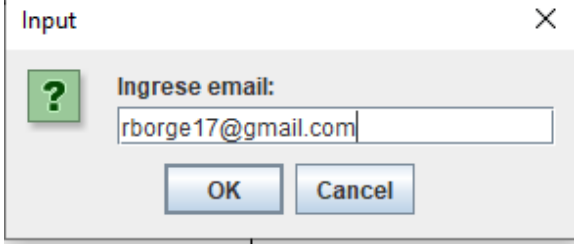
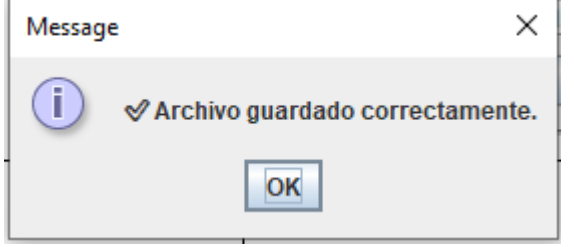
Después de cada venta, cuando se haya reducido la cantidad de un producto, se guarda el inventario actualizado de nuevo en el archivo utilizando guardarProductos().

```
public void guardarProductos() {  
    try (BufferedWriter writer = new BufferedWriter(new FileWriter("productos.txt"))) {  
        for (Producto producto : inventario.values()) {  
            // Guardar cada producto en el archivo con sus datos  
            writer.write(producto.getCodigo() + "," + producto.getNombre() + "," + producto.getPrecio() + "," + producto.getCantidad() + "," + producto.getCategoria());  
            writer.newLine();  
        }  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

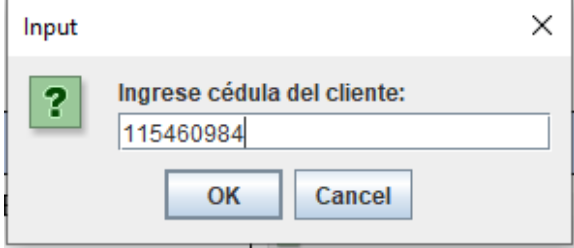
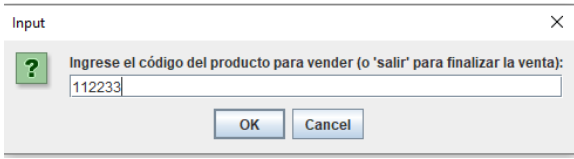
## Ejemplos de la gestión de ventas

primero para poder realizar una venta debemos registrar al cliente.

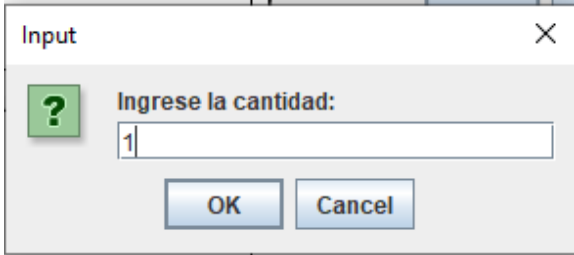
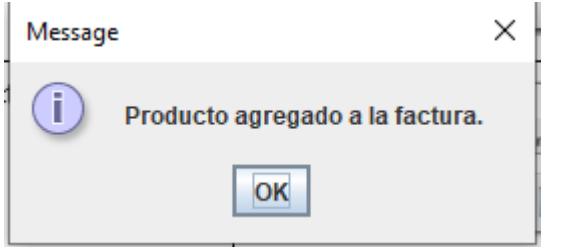
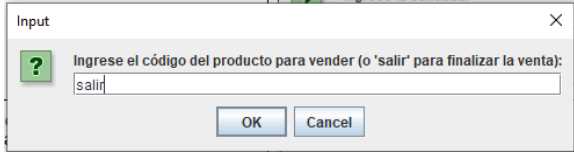
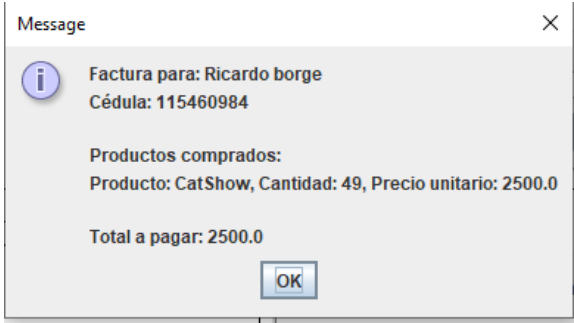
| Acción                                             | Imagen de Ejemplo                                                                    |
|----------------------------------------------------|--------------------------------------------------------------------------------------|
| Menú principal para GERENTE                        |   |
| Menú al ingresar en el botón de Gestionar Clientes |  |
| Agregar Cliente                                    |  |
| Agregar Apellidos                                  |  |

|                          |                                                                                     |
|--------------------------|-------------------------------------------------------------------------------------|
| Agregar número de cédula |   |
| Agregar Correo           |   |
| Confirmación de guardado |  |

## Gestión de la venta

| Acción                                                                                                  | Imagen de Ejemplo                                                                    |
|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Menú principal para GERENTE                                                                             |  |
| Al ingresar en el botón de procesar ventas nos pide que ingresemos la cédula del cliente ya registrado. |  |
| solicita el código del producto                                                                         |  |



|                                                                                                      |                                                                                     |
|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Solicita la cantidad                                                                                 |   |
| confirmación de que el producto fue agregado a la factura                                            |   |
| le ponemos salir para finalizar la venta                                                             |   |
| Se muestra la factura con la información del cliente y la cantidad de producto restante en el local. |  |

## Clase CargaInformacion

```
public class CargaInformacion {
    private static HashMap<String, Usuario> todosLosUsuarios = new HashMap<>();
    private static HashMap<String, Producto> inventario = new HashMap<>();
    private static HashMap<String, Cliente> clientes = new HashMap<>();
    private FastOptionPane fop = new FastOptionPane();
    //private Inventario inventarioValidaciones = new Inventario();

    public void cargarUsuarios() {
        try (BufferedReader reader = new BufferedReader(new FileReader("usuarios.txt"))) {
            String line;
            while ((line = reader.readLine()) != null) {
                String[] datos = line.split(",");
                todosLosUsuarios.put(datos[0], new Usuario(datos[0], datos[1], datos[2], datos[3], datos[4], Rol.valueOf(datos[5])));
            }
        } catch (IOException e) {
            //fop.dialogoAlerta("Carga Usuarios", "Error al cargar los datos de los usuarios: " + e.getMessage());
            e.getMessage();
        }
    }

    public void guardarUsuarios() {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter("usuarios.txt"))) {
            for (Usuario usuario : todosLosUsuarios.values()) {
                // Guardar cada producto en el archivo con sus datos
                writer.write(usuario.getNombreUsuario() + "," + usuario.getNombre() + "," +
                    usuario.getApellidos() + "," + usuario.getCedula() + "," +
                    usuario.getContraseña() + "," + usuario.getRol());
                writer.newLine();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public HashMap<String, Usuario> getUsuariosCargados () {
        return todosLosUsuarios;
    }
}
```

Esta clase es la encargada de obtener la información de los diferentes archivos y administrarlos según convenga. Dentro de ella, existen los métodos de “cargar” los cuales son los encargados de abrir los diferentes archivos donde se encuentra la información y se carga en los hashmaps. Luego, para poder obtener esa información cargada, se debe usar un get para que la clase pase el hashmap completo a la clase que requiere la información. A su vez, existen los métodos de “guardar” los cuales permiten transferir la información de los hashmaps a los archivos para así mantener la base de datos actualizada. En el ejemplo de la imagen, se puede observar el código específicamente para los Usuarios, pero tanto Productos como Clientes también usan la misma lógica para la carga y guardado de información.

## Métodos definidos para la Clase CargaInformacion

| Método                 | Uso                                                                         |
|------------------------|-----------------------------------------------------------------------------|
| cargarUsuarios()       | Carga información del archivo "usuarios.txt" al hashmap "todosLosUsuarios"  |
| guardarUsuarios()      | Guarda información del hashmap "todosLosUsuarios" al archivo "usuarios.txt" |
| getUsuariosCargados()  | Retorna el hashmap "todosLosUsuarios"                                       |
| cargarProductos()      | Carga información del archivo "productos.txt" al hashmap "inventario"       |
| guardarProductos()     | Guarda información del hashmap "inventario" al archivo "productos.txt"      |
| getInventarioCargado() | Retorna el hashmap "inventario"                                             |
| cargarClientes()       | Carga información del archivo "clientes.txt" al hashmap "clientes"          |
| guardarClientes()      | Guarda información del hashmap "clientes" al archivo "clientes.txt"         |
| getClientesCargados    | Retorna el hashmap "clientes"                                               |

## Clase Cliente y ClienteGestiones

```
public class Cliente {
    private String nombre;
    private String apellidos;
    private String cedula;
    private String email;

    public Cliente(String nombre, String apellidos, String cedula, String email) {
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.cedula = cedula;
        this.email = email;
    }

    public String getNombre() { return nombre; }
    public String getApellidos() { return apellidos; }
    public String getCedula() { return cedula; }
    public String getEmail() { return email; }

    public void setNombre(String nombre) { this.nombre = nombre; }
    public void setApellidos(String apellidos) { this.apellidos = apellidos; }
    public void setEmail(String email) { this.email = email; }
}
```

La clase Cliente simplemente define un arreglo de objetos de tipo Cliente, donde cada valor necesario para el cliente, es almacenado. Luego se definen los gets y sets para encapsular apropiadamente.

```
public class ClienteGestiones {
    private static HashMap<String, Cliente> clientes = new HashMap<>();
    private CargaInformacion carga = new CargaInformacion();
    private FastOptionPane fop = new FastOptionPane();

    private String nuevoNombre, precioString, cantidadString;

    private JTextField campoNombreC = new JTextField();
    private JTextField campoApellidosC = new JTextField();
    private JTextField campoCedulaC = new JTextField();
    private JTextField campoEmailC = new JTextField();

    public void inicializarClientes(){
        carga.cargarClientes();
        clientes = carga.getClientesCargados();
    }

    public void menuGestionarClientes() { ...40 lines }

    public Cliente buscarCliente(String cedula) {
        Cliente cliente = clientes.get(cedula);
        if (cliente != null && cliente.getCedula().equals(cedula)) {
            return cliente;
        } else {
            return null;
        }
    }

    public void agregarCliente(){
        Object[] opcionesCliente = new Object[]{"Por favor, ingrese los datos del Cliente", " ", "Nombre del Cliente", campoNombreC, "Apellidos del Cliente", campoApellidosC,
        "Cedula del Cliente", campoCedulaC, "Correo electronico", campoEmailC};
        int confirm = fop.dialogoOkCancel("Agregar Nuevo Cliente", opcionesCliente);
        if (confirm == 0){
            String nombre = campoNombreC.getText();
            String apellidos = campoApellidosC.getText();
            String cedula = campoCedulaC.getText();
            String email = campoEmailC.getText();
            Cliente nuevoCliente = new Cliente(nombre, apellidos, cedula, email);
            clientes.putIfAbsent(nuevoCliente.getCedula(), nuevoCliente);
            fop.dialogoInformacion("Agregar Nuevo Cliente", "Cliente agregado satisfactoriamente.");
            carga.guardarProductos();
        }
    }
}
```

```

public void listarTablaClientes() {
    if (clientes.isEmpty()) {
        JOptionPane.showMessageDialog(null, "La lista de clientes está vacía.", "Aviso", JOptionPane.INFORMATION_MESSAGE);
    } else {
        // Columnas y datos de ejemplo
        String[] columnas = {"Nombre", "Apellido", "Cédula", "Correo"};

        // Crear el arreglo con las dimensiones adecuadas
        Object[][] datos = new Object[clientes.size()][columnas.length]; // 3 columnas: cédula, nombre, correo

        // Llenar el arreglo
        for (int i = 0; i < clientes.size(); i++) {
            Cliente c = clientes.get(i);
            for (int j = 0; j < columnas.length; j++) {
                switch (j) {
                    case 0:
                        datos[i][j] = c.getNombre();
                        break;
                    case 1:
                        datos[i][j] = c.getApellidos();
                        break;
                    case 2:
                        datos[i][j] = c.getCedula();
                        break;
                    case 3:
                        datos[i][j] = c.getEmail();
                        break;
                }
            }
        }

        // Crear el modelo y la tabla
        DefaultTableModel modelo = new DefaultTableModel(datos, columnas);
        JTable tabla = new JTable(modelo);
        tabla.setEnabled(false); // Opcional: hacerla solo de lectura

        // Meter la tabla en un JScrollPane para scroll
        JScrollPane scrollPane = new JScrollPane(tabla);
        scrollPane.setPreferredSize(new java.awt.Dimension(400, 100));

        // Mostrarlo en un JOptionPane
        JOptionPane.showMessageDialog(null, scrollPane, "Lista de Clientes", JOptionPane.INFORMATION_MESSAGE);
    }
}

```

La clase ClienteGestiones es la encargada de tener el menú de gestión de clientes y los diferentes métodos de cada una de las opciones. Como estamos haciendo uso de hashmaps, es muy sencillo obtener el objeto que se necesita, ya que utilizamos valores únicos como llaves (cedula, usuario, código), lo que nos permite rápidamente identificar existencias y extracción de datos.

## Metodos definidos en ClienteGestiones

| Metodo                | Uso                                                                                                                                      |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| inicializarClientes() | Llama al método de carga de la clase "CargaInformacion" relacionado con los clientes, obteniendo el hashmap actualizado de los clientes. |
| menuGestionClientes() | Despliega el menú de gestión de clientes y lleva a los métodos adecuados.                                                                |
| buscarCliente()       | Dada una cedula, retorna el cliente asociado a dicha cedula.                                                                             |
| agregarCliente()      | Genera un nuevo objeto en el hashmap "clientes" de tipo Cliente.                                                                         |
| modificarCliente()    | Si existe el cliente, permite modificar cualquiera de los valores que tenga el objeto y lo actualiza en el hashmap.                      |
| eliminarCliente()     | Si existe el cliente, permite eliminar el objeto asociado a dicho cliente.                                                               |
| listarTabaClientes()  | Muestra en pantalla una tabla con los clientes.                                                                                          |

## Clase Usuario y UsuarioGestiones

La clase Usuario es la encargada de administrar las características de cada usuario que exista. Como estamos usando HashMaps, esta clase es la encargada de almacenar las características del usuario que hizo login. Como observamos, las características del usuario están encapsuladas por medio de la definición de variables tipo private, y dichas variables son inicializadas por el constructor cuando se llame a la clase, pasándole los argumentos correspondientes. Para poder obtener los valores de cada variable, se declaran métodos get, para que las demás clases solamente puedan acceder dicha información a través del método, así no son modificables.

```
package sistemaventaspetsmarket;

public class Usuario {
    private String nombreUsuario;
    private String nombre;
    private String apellidos;
    private String cedula;
    private String contraseña;
    private Rol rol;

    public Usuario(String nombre, String nombreUsuario, String apellidos, String cedula, String contraseña, Rol rol) {
        this.nombreUsuario = nombreUsuario;
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.cedula = cedula;
        this.contraseña = contraseña;
        this.rol = rol;
    }

    public String getNombre() {
        return nombre;
    }

    public String getNombreUsuario() {
        return nombreUsuario;
    }

    public String getApellidos() {
        return apellidos;
    }

    public String getCedula() {
        return cedula;
    }

    public String getContrasena() {
        return contraseña;
    }

    public Rol getRol() {
        return rol;
    }
}
```

```

1 //
2 public class UsuarioGestiones {
3
4     private static HashMap<String, Usuario> usuarios = new HashMap<>();
5     private CargaInformacion carga = new CargaInformacion();
6
7     public void inicializarUsuarios() {
8         carga.cargarUsuarios();
9         usuarios = carga.getUsuariosCargados();
10    }
11
12    public HashMap getUsuarios() {
13        return usuarios;
14    }
15 }

```

## Métodos definidos en UsuarioGestiones

| Metodo                | Uso                                                                                                                                      |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| inicializarUsuarios() | Llama al método de carga de la clase "CargaInformacion" relacionado con los usuarios, obteniendo el hashmap actualizado de los usuarios. |
| menuGestionUsuarios() | Despliega el menú de gestión de usuarios y lleva a los métodos adecuados.                                                                |
| buscarUsuario()       | Dado un usuario, retorna el cliente asociado a dicho usuario.                                                                            |
| agregarUsuario()      | Genera un nuevo objeto en el hashmap "usuarios" de tipo Usuario.                                                                         |
| modificarUsuario()    | Si existe el usuario, permite modificar cualquiera de los valores que tenga el objeto y lo actualiza en el hashmap.                      |
| eliminarUsuario()     | Si existe el usuario, permite eliminar el objeto asociado a dicho usuario.                                                               |
| listarTabaUsuarios()  | Muestra en pantalla una tabla con los usuarios.                                                                                          |



# Clase Sistema

A nivel del sistema principal, se modifico para trasladar la mayoría de los metodos a sus respectivas clases, los que ahora llamamos “Gestiones”, para evitar sobrecargar todo en la misma clase. Redujimos unicamente al menu principal y al inicio de sesion.

```
public void iniciarSesion() {
    boolean activo = true;
    while (activo) {
        String[] opcionesInicio = new String[]{"Iniciar Session", "Salir"};
        int selec = fop.dialogoOpciones(opcionesInicio, "PetsMarket", "-- Bienvenido a PetsMarket --\n" + "Por seguridad debe iniciar sesión");
        if (selec == 0) {
            boolean activo2 = true;
            while (activo2) {
                Object[] mensaje = new Object[]{"Ingrese el Usuario y la Contraseña", " ", "Usuario", CampoUsuario, "Contraseña", CampoPassword};
                int confirm = fop.dialogoOkCancel("PetsMarket", mensaje);
                if (confirm == 0) {
                    Usuario usuario = val.validarUsuario(CampoUsuario.getText(), new String(CampoPassword.getPassword()));
                    if (usuario != null) {
                        fop.dialogoInformacion("PetsMarket", "Inicio de sesión exitoso.\nBienvenido, " + usuario.getNombre() + "!");
                        try {
                            menuPrincipal(usuario);
                            activo2 = false;
                        } catch (IOException e) {
                            e.printStackTrace();
                        }
                    } else {
                        fop.dialogoAlerta("PetsMarket", "Error. Usuario o contraseña incorrectos");
                        int opc = fop.dialogoConfirmacion("PetsMarket", "¿Desea intentarlo de nuevo?");
                        if (opc != 0) {
                            fop.dialogoInformacion("PetsMarket", "Saliendo al inicio...");
                            activo2 = false;
                        }
                    }
                } else {
                    fop.dialogoInformacion("PetsMarket", "Saliendo al inicio...");
                    activo2 = false;
                }
            }
        } else {
            fop.dialogoInformacion("PetsMarket", "Se va cerrar el sistema...");
            activo = false;
        }
    }
}
//fin del primer while
}
//fin de iniciar sesion
```

```
public class Sistema {

    private FastOptionPane fop = new FastOptionPane();
    private Validaciones val = new Validaciones();
    private Inventario inventario = new Inventario();
    //private ClienteGestiones clientes = new ClienteGestiones();
    private UsuarioGestiones usuarios = new UsuarioGestiones();

    private ArrayList<Cliente> clientes = new ArrayList<>();

    private JTextField CampoUsuario = new JTextField();
    private JPasswordField CampoPassword = new JPasswordField();

    public void iniciarSesion() { ...42 lines } //fin de iniciar sesion

    public void menuPrincipal(Usuario usuario) throws IOException {
        iniciar();
        //boolean cerrarSistema = true;
        boolean activo = true;
        String[] opcionesMenu;
        if (usuario.getRol() == Rol.GERENTE) {
            opcionesMenu = new String[]{"Gestionar Inventario", "Gestionar Clientes", "Procesar Ventas", "Gestionar Usuarios", "Cerrar Sesión"};
        } else {
            opcionesMenu = new String[]{"Gestionar Inventario", "Gestionar Clientes", "Procesar Ventas", "Cerrar Sesión"};
        }
        while (activo) {
            String mensaje = "Menú Principal - PetsMarket\nNombre: " + usuario.getNombre() + "\nUsuario: " + usuario.getNombreUsuario() + " (" + usuario.getRol() + ")\n\nSeleccione una opción:";
            int seleccion = fop.dialogoOpciones(opcionesMenu, "Menu Principal", mensaje);

            if (seleccion == JOptionPane.CLOSED_OPTION) break;
            switch (seleccion) {
                case 0:
                    inventario.menuGestionarInventario();
                    break;
                case 1:
                    gestionarClientes();
                    break;
                case 2:
                    realizarVenta(usuario.getNombreUsuario());
                    break;
                case 3:
                    if (usuario.getRol() == Rol.GERENTE) {
                        gestionUsuarios();
                    }
            }
        }
    }
}
```

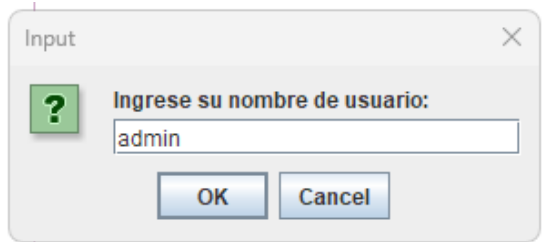
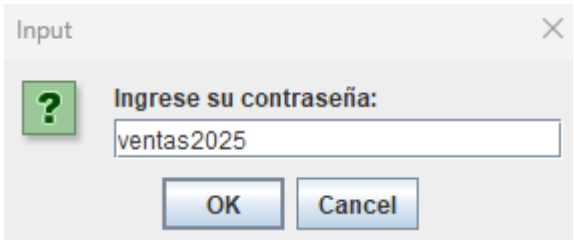
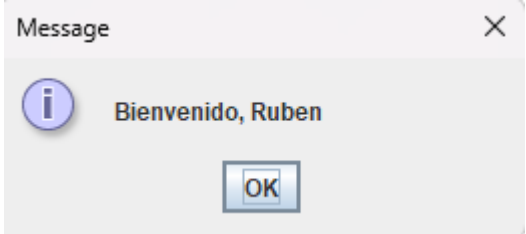
## Metodos definidos en Clase Sistema

| Metodo          | Uso                                                                                                           |
|-----------------|---------------------------------------------------------------------------------------------------------------|
| iniciarSesion() | Muestra el menu de inicio de sesión y controla el ingreso de los usuarios.                                    |
| menuPrincipal() | Despliega el menú principal del sistema que va a llevar a cada uno de los menús específicos de las gestiones. |

## Ejemplos del uso del sistema

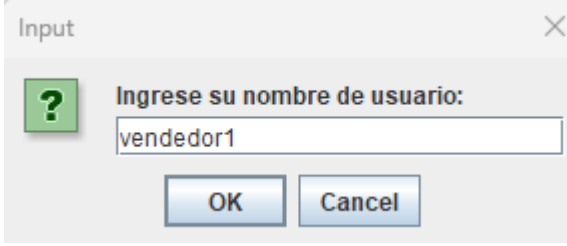
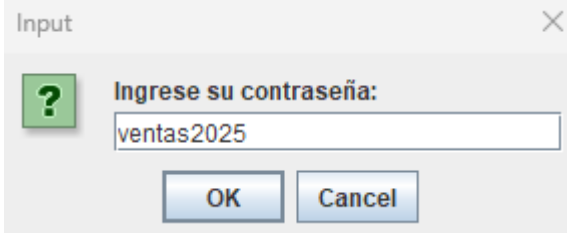
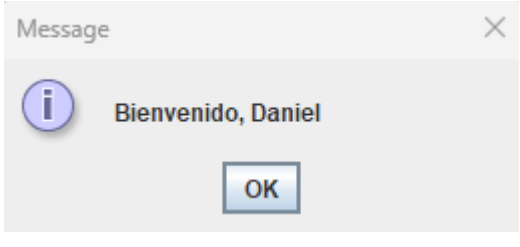
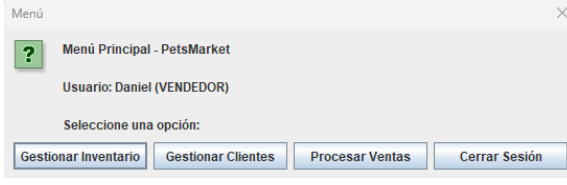
Ya implementado lo anterior, podemos mostrar unos ejemplos de cómo podemos ingresar, y según el rol del Usuario, así será el menú que obtiene.

### Ejemplo Usuario tipo GERENTE

| Acción                                                              | Imagen de Ejemplo                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ingreso de usuario tipo VENDEDOR                                    |  A screenshot of a Windows-style 'Input' dialog box. It has a title bar with a close button. Inside, there's a green question mark icon, the text 'Ingrese su nombre de usuario:', a text input field containing 'admin', and two buttons at the bottom: 'OK' and 'Cancel'.                                                                                                        |
| Ingreso de contraseña                                               |  A screenshot of a Windows-style 'Input' dialog box. It has a title bar with a close button. Inside, there's a green question mark icon, the text 'Ingrese su contraseña:', a text input field containing 'ventas2025', and two buttons at the bottom: 'OK' and 'Cancel'.                                                                                                         |
| Contraseña correcta                                                 |  A screenshot of a Windows-style 'Message' dialog box. It has a title bar with a close button. Inside, there's a blue information icon, the text 'Bienvenido, Ruben', and an 'OK' button at the bottom.                                                                                                                                                                           |
| Menú Principal con opciones limitadas para el usuario tipo VENDEDOR |  A screenshot of a Windows-style 'Menu' dialog box. It has a title bar with a close button. Inside, there's a green question mark icon, the text 'Menú Principal - PetsMarket', 'Usuario: Ruben (GERENTE)', 'Seleccione una opción:', and five buttons at the bottom: 'Gestionar Inventario', 'Gestionar Clientes', 'Procesar Ventas', 'Gestionar Usuarios', and 'Cerrar Sesión'. |



## Ejemplo Usuario tipo VENDEDOR

| Acción                                                              | Imagen de Ejemplo                                                                    |
|---------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Ingreso de usuario tipo VENDEDOR                                    |    |
| Ingreso de contraseña                                               |    |
| Contraseña correcta                                                 |   |
| Menú Principal con opciones limitadas para el usuario tipo VENDEDOR |  |

# Clase FastOptionPane

```
import javax.swing.JOptionPane;
import javax.swing.JScrollPane;
import javax.swing.JTable;

public class FastOptionPane {

    public String Input_Dialog (String titulo, String mensaje) {
        String value = JOptionPane.showInputDialog(null, mensaje, titulo, JOptionPane.INFORMATION_MESSAGE);
        return value;
    }

    public void dialogoInformacion (String titulo, String mensaje){
        JOptionPane.showMessageDialog(null, mensaje, titulo, JOptionPane.INFORMATION_MESSAGE);
    }

    public void dialogoAlerta (String titulo, String mensaje){
        JOptionPane.showMessageDialog(null, mensaje, titulo, JOptionPane.WARNING_MESSAGE);
    }

    public int dialogoConfirmacion(String titulo, String mensaje){
        int confirm = JOptionPane.showConfirmDialog(null, mensaje, titulo, JOptionPane.YES_NO_OPTION, JOptionPane.QUESTION_MESSAGE);
        return confirm;
    }

    public int dialogoOpciones (String op1, String op2, String titulo, String mensaje){
        Object[] opciones JOptionPane = {op1, op2};
        int opc = JOptionPane.showOptionDialog (null, mensaje, titulo, JOptionPane.YES_NO_OPTION, JOptionPane.QUESTION_MESSAGE, null, opciones_JOptionPane, opciones_JOptionPane[0]);
        return opc;
    }

    public void mostrarTabla (String titulo, Object[][] files, Object[] columnas){
        JTable table = new JTable(files, columnas);
        JOptionPane.showMessageDialog(null, new JScrollPane(table));
    }

}
```

Adicionalmente a las clases que se tienen, se va a implementar una clase para facilitar el uso de JOptionPane. Buscamos unificar los diferentes JOptionPane que vamos a usar en el proyecto para que cada vez que necesitemos pedir o mostrar información, sea más rápido y más fácil usar un objeto, cuyos métodos reciben diferentes atributos, que luego serán implementados en los diferentes JOptionPane que conocemos, así no hay que estar modificando constantemente cada JOptionPane.

Inicialmente tenemos 6 métodos, pero al ir avanzando en el proyecto, se agregarán más o se modificarán los que ya existen para que sean más genéricos.

# Link al video de presentación

<https://youtu.be/TBHuj10Lpr8>